Atividade: aplicação web mínima, em duas etapas

Individual · Etapa 1 (proposta do app): até quarta, 21/10/2026, 18h30 · Etapa 2 (o app): até quarta, 28/10/2026, 23h59

1. Objetivo

Construir uma aplicação web pequena e simples, do tipo **campo de entrada, botão e área de resultado**, planejando-a antes por escrito. A **Etapa 1** é uma proposta curta do app (o que ele faz e como será usado). A **Etapa 2** é o app funcionando, com o código publicado. O app deve ser compatível com o prazo de uma semana: o que vale é o básico bem feito, não a complexidade.

2. Escolha do tema

1. Quiz de revisão	2. Lista de tarefas	3. Conversor de unidades
4. Painel de hábitos	5. Flashcards	6. Simulador de entrevista
7. Diário de humor	8. Calculadora de projetos	9. Planejador semanal
10. Controle de gastos	11. Resumidor de texto	12. Gerador de nomes

Pode propor outro tema com a mesma estrutura (entrada, botão, resultado); basta descrevê-lo na proposta.

3. Etapa 1: proposta do app (1 página, em PDF)

A proposta é uma descrição curta do que será feito, sem a complexidade de um documento de requisitos completo. Deve conter:

1. **Nome e tema** do app.
2. **Descrição:** o que o app faz e para quem (3 a 4 linhas).
3. **Exemplo de uso:** um caso concreto, passo a passo, com um valor de entrada real e o resultado esperado.
4. **O que o app faz** (3 a 5 itens) **e o que não faz** (2 a 3 itens).
5. **Como funciona:** o que acontece do clique no botão até o resultado aparecer (3 a 4 passos).
6. **Tecnologia:** trilha escolhida (seção 5) e, se houver, a API que pretende usar.
7. **Cuidados:** 1 ou 2 riscos do app (por exemplo, o que pode dar errado se o usuário digitar algo inesperado) e como pretende evitá-los.
8. **Uso de IA:** se pretende usar, qual ferramenta e para quê.

Exemplo (tema “Lista de tarefas”). Descrição: o app ajuda estudantes a anotar tarefas do dia e marcá-las como concluídas. Exemplo de uso: o usuário digita “Estudar HTTP”, clica em Adicionar e a tarefa aparece na lista; ao clicar nela, fica riscada. Faz: adiciona, conclui e remove tarefas; guarda a lista no navegador. Não faz: login, sincronização entre aparelhos. Como funciona: o clique dispara uma função JavaScript que lê o campo, cria o item e salva no `localStorage`. Trilha: C (sem API). Cuidados: texto digitado entra na página com `textContent`; campo vazio mostra aviso. IA: nenhuma.

4. Etapa 2: o app

Requisitos mínimos:

- Página com campo de entrada, botão e área de resultado, em HTML, CSS e JavaScript, sem frameworks.
- Funciona do início ao fim, inclusive com entrada vazia (mostra uma mensagem, sem quebrar).
- HTML semântico (`header`, `main`, `label` ligado ao campo etc.) e CSS responsivo (legível no celular e no computador).
- Código organizado: funções com nomes claros, sem repetição desnecessária, comentários onde o código não for óbvio.
- Dado digitado ou vindo de uma API entra na página com `textContent`, e não com `innerHTML`.
- Código em repositório público no GitHub, com **README** (o que o app faz, como rodar, uso de IA) e **commits** feitos ao longo do trabalho, com mensagens descritivas.
- Nenhuma chave de API no repositório nem no histórico de commits.

- No README, o **print da aba Rede** do navegador (veja a seção 6) com uma explicação de 2 a 3 linhas do que ele mostra.

5. Trilhas (todas valem igual)

- **A, API de IA:** a saída é gerada por um serviço de inteligência artificial (exige conta e chave de API; siga a documentação do provedor).
- **B, API pública:** a saída vem de um serviço aberto (por exemplo, consulta de CEP ou clima). Confira a documentação da API.
- **C, sem API:** a lógica fica no código e os dados no `localStorage` do navegador.
- **Plano B** (sem internet ou API fora do ar): usar uma resposta pronta guardada em um arquivo `.js` do projeto e declarar isso no README.

Na trilha A, a chave fica em um arquivo `config.js` listado no `.gitignore`, nunca no GitHub. Se uma chave vazar, revogue-a e crie outra. Não publique (por exemplo, no GitHub Pages) uma página que contenha chave.

6. Como rodar e capturar a aba Rede

Rode a página por um servidor local: no VS Code, a extensão Live Server; ou, no terminal da pasta, `python -m http.server 8000` e abra `http://localhost:8000`. Pressione F12, abra a aba **Rede** (*Network*), recarregue (Ctrl+R) e use o app. Clique na requisição e capture a tela mostrando **método** (GET ou POST), **URL** e **status** (200, 404 etc.). Trilhas A e B: a requisição à API. Trilha C: a requisição que carregou a página.

7. O que será avaliado

Etapa 1: proposta	Etapa 2: app
Descrição clara e específica (para quem e para quê) Exemplo de uso concreto, com entrada e resultado Escopo realista para uma semana (faz e não faz) Explicação do funcionamento e dos cuidados Uso de IA declarado	Funcionamento do início ao fim HTML semântico e responsividade Qualidade e organização do código Repositório: README e commits Segurança básica (sem chave exposta, `textContent`) Coerência com a proposta e aba Rede explicada Uso de IA declarado e capacidade de explicar o que entregou

A pontuação de cada etapa na P1 será informada em aula.

8. Uso de IA

É permitido e deve ser declarado (no README e na proposta): a ferramenta, o que foi pedido e o que você alterou. Declarar não reduz a nota. Você deve saber explicar o que entregou.

9. Entrega

Etapa 1: proposta digitada (preencha o modelo da última página no computador e envie em PDF), por e-mail para **rs.freire@ufma.br**, assunto `ATV19/10 DSW - Nome - matrícula`, até quarta 21/10, 18h30.
Etapa 2: para o mesmo endereço, em nova mensagem, assunto `ATV19/10 DSW Etapa 2 - Nome - matrícula`, com o link do repositório, até quarta 28/10, 23h59.

Universidade Federal do Maranhão · Engenharia da Computação · Desenvolvimento de Sistemas Web (EECP0036) · Turma 24N12 · 2026.2 · Prof. Renan Sousa Freire

Modelo da proposta do app (Etapa 1)

Preencha direto no PDF (clique nas caixas e digite) ou use o modelo em Word. Não precisa imprimir.

Aluno	Matrícula	Tema	Trilha (A, B ou C)
Wendel Davi Reis Costa	2023035735	11. Resumidor de texto	A

1. Nome do app e descrição

O que faz e para quem (3 a 4 linhas).

O Sintetiza Hub é uma aplicação web voltada principalmente a estudantes que precisam compreender e revisar textos extensos. O usuário insere um texto e solicita sua sumarização por meio de inteligência artificial. O aplicativo apresenta um resumo com as principais ideias do conteúdo, utilizando uma linguagem clara e objetiva.

2. Exemplo de uso

Um caso concreto, passo a passo: o que o usuário digita e o que aparece.

Entrada do usuário:

"A energia solar é obtida a partir da luz e do calor do Sol. Por meio de painéis fotovoltaicos, essa energia pode ser convertida em eletricidade para residências, empresas e indústrias. Além de ser uma fonte renovável, contribui para reduzir a dependência de combustíveis fósseis. Apesar do investimento inicial para instalação, os sistemas podem diminuir os gastos com eletricidade ao longo do tempo."

3. O que o app faz e o que não faz

Faz: 3 a 5 itens. Não faz: 2 a 3 itens.

Faz:
- Recebe textos digitados ou colados pelo usuário.
- Gera resumos utilizando inteligência artificial.
- Identifica e apresenta as principais ideias do conteúdo.

4. Como funciona

Do clique no botão até o resultado aparecer (3 a 4 passos).

1. O usuário insere ou cola um texto no campo de entrada e clica em "Gerar resumo".
2. O JavaScript verifica se o campo foi preenchido corretamente e prepara o conteúdo para envio.
3. O texto é enviado à API do Google Gemini, que processa as informações e gera um resumo.
4. O aplicativo recebe a resposta e apresenta o resultado na página, utilizando textContent para exibir o

5. Tecnologia

Trilha escolhida e API, se houver.

Trilha A — API de Inteligência Artificial.

6. Cuidados

1 ou 2 riscos e como pretende evitá-los.

A aplicação evitará o risco de injeção de código malicioso (XSS) utilizando textContent para renderizar o conteúdo recebido, o que impedirá que os textos inseridos sejam interpretados como HTML. Para evitar o risco de exposição de credenciais, a chave da API ficará isolada em um arquivo config.js incluído no .gitignore, garantindo que não seja enviada ao GitHub. Adicionalmente, para evitar instabilidades e falhas na experiência do

7. Uso de IA

Ferramenta e para quê, ou “nenhum”.

Será utilizada a API Google Gemini para gerar os resumos dos textos inseridos. Também será utilizado o ChatGPT como ferramenta de apoio na elaboração da proposta, organização dos arquivos e desenvolvimento do código. As sugestões de implementação serão revisadas, testadas e ajustadas durante o desenvolvimento